1. Linear search: trace and explain

Name: _____ Class: _____ Date: _____

WAGBA: Follow and explain search algorithms so we can choose the correct steps to find an item.

Start with Questions 1-5. Mr Dave will tell you which follow-up questions to do. Question 11 is an optional challenge.

The librarian needs to find **target 13**. Follow the supplied linear-search algorithm in order. Do not jump straight to the answer you can see.

Index	0	1	2	3	4	5	6
Stored value	**5**	**9**	**13**	**18**	**24**	**31**	**42**

Algorithm reference: line numbers help you locate instructions.

```
01  position ← 0
02  found ← False
03  WHILE position < LEN(items) AND found = False
04      IF items[position] = target THEN
05          found ← True
06      ELSE
07          position ← position + 1
08      ENDIF
09  ENDWHILE
10  OUTPUT found
```

Read the symbols: ← stores a value; = compares values. **found** is True or False. The algorithm outputs **found**, not the matching index.

1. Which values will linear search check to find 13? Complete the two blank rows below.

The first row is a worked example. In each row record the index, its stored value, whether it equals 13, and what the algorithm does next.

Index (position)	Stored value	Equals target 13?	Next action
0	5	No	Set position to 1

2. Why does this search stop after finding 13? State the final value of found and what line 10 outputs.

Optional writing prompt: found becomes __. Line 10 outputs __. The WHILE loop stops because __.

---

3. A student checks value 5 and immediately reports "not found". Why is this too early, and which index should the search check next?

---

2. Binary search: read and trace

Name: _____ Class: _____ Date: _____

WAGBA: Follow and explain search algorithms so we can choose the correct steps to find an item.

Now search for **target 13** using binary search. The list is sorted from smallest to largest. Index labels stay fixed; no items are deleted or renumbered.

Index	0	1	2	3	4	5	6
Stored value	**5**	**9**	**13**	**18**	**24**	**31**	**42**

low, high and mid are indexes (positions). items[mid] is the value stored at the midpoint index. DIV 2 uses whole-number division.

```
01  low ← 0
02  high ← LEN(items) − 1
03  found ← False
04  WHILE low ≤ high AND found = False
05      mid ← (low + high) DIV 2
06      IF items[mid] = target THEN
07          found ← True
08      ELSE
09          IF items[mid] < target THEN
10              low ← mid + 1
11          ELSE
12              high ← mid − 1
13          ENDIF
14      ENDIF
15  ENDWHILE
16  OUTPUT found
```

4. Which items does binary search check to find 13? Complete the two blank rows in the trace.

The first row is a worked example. Record low, high and mid **before** changing a boundary. Write the stored value at mid. In "Next action", state the boundary update or set found to True.

low (first index)	high (last index)	mid (index)	Value at items[mid]	Next action
0	6	3	18	Set high to 2; keep indexes 0-2

Check before turning over: the index columns contain positions, not book numbers. On the next page you will explain why the boundaries changed.

3. Binary search: explain each decision

Name: _____ Class: _____ Date: _____

WAGBA: Follow and explain search algorithms so we can choose the correct steps to find an item.

Use the same sorted list. Give both a decision and a reason. Complete Question 5 first; Questions 6 and 7 are follow-up practice when your teacher asks.

Index	0	1	2	3	4	5	6
Stored value	**5**	**9**	**13**	**18**	**24**	**31**	**42**

5. For target 13, the first check uses midpoint index 3, storing value 18. What should the new last index (high) be, and why?

Optional writing prompt: The target value __ is smaller than __. Sorted order rules out indexes __ to __. The remaining indexes are __ to __, so high becomes __.

6. For target 31, binary search checks value 18 at index 3. A student writes low ← mid. Why should index 3 be excluded, and what should the new first index (low) be?

Optional writing prompt: Index 3 cannot match because __. The next possible index is __, so low becomes __.

7. For target 20, the algorithm has checked 18, 31 and 24. It now has low = 4 and high = 3 (both indexes). What does this tell you about the possible search positions? State found and the output on line 16.

Optional writing prompt: No possible __ remain because __. found is __, so line 16 outputs __.

4. Compare the two searches

Name: _____ Class: _____ Date: _____

WAGBA: Follow and explain search algorithms so we can choose the correct steps to find an item.

Follow-up when your teacher asks: search for **target 42** using both algorithms on pages 1 and 2. Both stop after finding a match.

Index	0	1	2	3	4	5	6
Stored value	**5**	**9**	**13**	**18**	**24**	**31**	**42**

Counting rule: one item check means inspecting one stored list value against the target. Do not count each individual operator or WHILE test.

8. Which values does linear search inspect, in order, including the matching value 42? Give the matching index and the total number of item checks.

Optional writing prompt: Checked values, in order: __. Matching index: __. Item checks: __.

9. Which positions and values does binary search inspect for target 42? Complete one row for each item check, then write its matching index and total checks.

Record the boundary indexes before updating them, as you did in Question 4.

low (index)	high (index)	mid (index)	Stored value	Next action

Matching index: _____ **Total item checks:** _____

10. Which algorithm uses fewer item checks for target 42? Use both counts and explain why sorted order helps binary search.

11 (optional challenge). Does binary search always use fewer item checks? Test that claim with target 5. List the checked values and counts for both searches, then give your conclusion.

Next step: Which specific step do you need to practise next? Use Question 1-11 as evidence. Complete your full KSU Learning Pit Stop on the website, or on paper if time is called.